Date:28/08/2026

3. k-fold cross validation

1. What is the K-Fold Cross-Validation method in machine learning, and how does it help in evaluating the performance of a model? Explain the steps involved in K-Fold Cross-Validation, its advantages, limitations.

2. Provide an example of implementation of k-fold cv in Python.

Procedure:

Step 1: Import the WDBC dataset directly into Python using the Pandas library.
Step 2: Assign the 30 features of the dataset to the input array X.
Step 3: Assign the diagnosis/class labels (M and B) to the target array y.
Step 4: Use LabelEncoder to convert the class labels into numerical values, where Malignant (M) = 1 and Benign (B) = 0**.**
Step 5: Divide the dataset into 80% training data and 20% testing data using train_test_split.
Step 6: Create a machine-learning pipeline containing StandardScaler, PCA, and Logistic Regression.
Step 7: Use StandardScaler to standardize the input features so that they are on a similar scale.
Step 8: Apply PCA (Principal Component Analysis) to reduce the 30 features to 2 principal components**.**
Step 9: Apply Logistic Regression to classify the tumors as malignant or benign.
Step 10: Train the pipeline using the training data with the fit() method.
Step 11: Evaluate the trained model using the testing data and calculate the test accuracy**.**
Step 12: Initialize Stratified K-Fold Cross-Validation with 10 folds to evaluate the model more reliably.
Step 13: Divide the training data into 10 folds and train the model using 9 folds while testing it on the remaining fold.
Step 14: Repeat the process for all 10 folds and calculate the accuracy for each fold.
Step 15: Calculate the mean and standard deviation of the 10 cross-validation accuracy scores.
Step 16: Finally, use cross_val_score() to perform the 10-fold cross-validation more efficiently and display the **CV** accuracy scores, average accuracy, and standard deviation.

Solution:

```
import pandas as pd
df = pd.read_csv("BHUMIKA B G SDM College(Autonomous), Ujire - wdbc (2).data", header=None)
print(df.head())
```

```
         0  1      2      3       4       5        6        7        8   \
0    842302  M  17.99  10.38  122.80  1001.0  0.11840  0.27760  0.3001
1    842517  M  20.57  17.77  132.90  1326.0  0.08474  0.07864  0.0869
2  84300903  M  19.69  21.25  130.00  1203.0  0.10960  0.15990  0.1974
3  84348301  M  11.42  20.38   77.58   386.1  0.14250  0.28390  0.2414
4  84358402  M  20.29  14.34  135.10  1297.0  0.10030  0.13280  0.1980

         9   ...     22     23      24      25      26      27      28      29  \
0  0.14710  ...  25.38  17.33  184.60  2019.0  0.1622  0.6656  0.7119  0.2654
1  0.07017  ...  24.99  23.41  158.80  1956.0  0.1238  0.1866  0.2416  0.1860
2  0.12790  ...  23.57  25.53  152.50  1709.0  0.1444  0.4245  0.4504  0.2430
3  0.10520  ...  14.91  26.50   98.87   567.7  0.2098  0.8663  0.6869  0.2575
4  0.10430  ...  22.54  16.67  152.20  1575.0  0.1374  0.2050  0.4000  0.1625

       30       31
0  0.4601  0.11890
1  0.2750  0.08902
2  0.3613  0.08758
3  0.6638  0.17300
4  0.2364  0.07678
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=1,
stratify=y)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (455, 30)
Testing data: (114, 30)
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

pipe_lr = Pipeline([('scl', StandardScaler()),
('pca', PCA(n_components=2)),('clf', LogisticRegression(random_state=1))])
pipe_lr.fit(X_train, y_train)
print('Test Accuracy: %.3f' % pipe_lr.score(X_test, y_test))
```

```
Test Accuracy: 0.956
```

```
import numpy as np
from sklearn.model_selection import StratifiedKFold

kfold = StratifiedKFold(n_splits=10,shuffle=True,random_state=1)
scores = []
for k, (train, test) in enumerate(kfold.split(X_train, y_train)):
    pipe_lr.fit(X_train[train], y_train[train])
    score = pipe_lr.score(X_train[test],y_train[test])
    scores.append(score)
    print('Fold: %2d, Class dist.: %s, Acc.: %.3f'
        % (k + 1,np.bincount(y_train[train]),score))
print('CV accuracy: %.3f +/- %.3f'
    % (np.mean(scores), np.std(scores)))
```

```
Fold:  1, Class dist.: [256 153], Acc.: 0.913
Fold:  2, Class dist.: [256 153], Acc.: 1.000
Fold:  3, Class dist.: [256 153], Acc.: 0.957
Fold:  4, Class dist.: [256 153], Acc.: 0.978
Fold:  5, Class dist.: [256 153], Acc.: 0.891
Fold:  6, Class dist.: [257 153], Acc.: 0.933
Fold:  7, Class dist.: [257 153], Acc.: 0.933
Fold:  8, Class dist.: [257 153], Acc.: 0.978
Fold:  9, Class dist.: [257 153], Acc.: 0.978
Fold: 10, Class dist.: [257 153], Acc.: 0.911
CV accuracy: 0.947 +/- 0.034
```